LAPORAN PROYEK · PORTOFOLIO UJI KOMPETENSI CLOUD COMPUTING ENGINEER

Laporan Proyek LabTrack

Perancangan, deployment, dan operasi sistem pencatatan praktikum laboratorium di Google Cloud

Disusun oleh: Muhammad Abrar Rayhan

Skema: Cloud Computing Engineer, SKKNI Nomor 456 Tahun 2015

Asesmen: Selasa, 27 Oktober 2026

Repositori: https://github.com/Muhammadabrarrayhan2/labtrack

Aplikasi web: https://labtrack-api-b26757zszq-et.a.run.app/app/

Dokumentasi API: https://labtrack-api-b26757zszq-et.a.run.app/docs

Versi aplikasi: 1.0.2 · Data pengujian diambil 26 Oktober 2026

Daftar Isi

Ringkasan eksekutif 4

1. Pendahuluan 5

    1.1 Latar belakang 5

    1.2 Rumusan masalah 5

    1.3 Tujuan 5

    1.4 Ruang lingkup 5

    1.5 Pemetaan ke unit kompetensi 6

2. Analisis kebutuhan 6

    2.1 Pengguna sistem 6

    2.2 Kebutuhan fungsional 7

    2.3 Kebutuhan nonfungsional 7

3. Perancangan sistem 8

    3.1 Arsitektur 8

    3.2 Pemilihan layanan Google Cloud 9

    3.3 Model data 9

    3.4 Keamanan 10

    3.5 Antarmuka web 10

4. Implementasi 11

    4.1 Struktur repositori 11

    4.2 Infrastruktur sebagai kode 12

    4.3 Pipeline CI/CD 12

    4.4 Pengujian otomatis 13

5. Deployment di Google Cloud 13

    5.1 Langkah deployment 13

    5.2 Hasil uji integrasi 14

6. Pengujian kinerja 15

    6.1 Uji beban 15

    6.2 Uji stress 16

6.3 Kesimpulan kapasitas 17
7. Operasi dan pemantauan 17
7.1 Aktivitas rutin 17
7.2 Pemantauan dan SLO 18
8. Dukungan teknis dan simulasi insiden 19
8.1 Alur dukungan berjenjang 19
8.2 Simulasi insiden 19
9. Biaya 20
10. Masalah yang ditemui dan penyelesaiannya 21
11. Penutup 21
11.1 Kesimpulan 21
11.2 Saran pengembangan 22
Lampiran 22
Daftar bukti 22
Tangkapan layar antarmuka web 23

Ringkasan eksekutif

LabTrack adalah sistem pencatatan praktikum untuk laboratorium kampus. Asisten memakainya untuk mengisi kehadiran setiap sesi, menerima laporan PDF dari mahasiswa, lalu memberi nilai. Koordinator memakai rekapnya untuk melihat siapa yang kehadirannya sudah di bawah batas sebelum ujian. Sebelum ada LabTrack, pekerjaan ini biasanya tersebar di lembar kerja yang disalin dari satu asisten ke asisten lain, dan laporan PDF dikirim lewat surel atau grup percakapan.

Proyek ini saya bangun sebagai bukti portofolio (APL02) untuk uji kompetensi skema Cloud Computing Engineer. Karena itu ukurannya sengaja dibuat realistis: satu layanan web yang benar-benar berjalan di Google Cloud, dipasang dengan Terraform, dirilis lewat GitHub Actions, dipantau dengan SLO, dan sudah melewati uji beban, simulasi insiden, serta uji pemulihan cadangan. Semua angka dalam laporan ini berasal dari pengujian di proyek `labtrack-1026146` pada 26 Oktober 2026. Kalau ada data contoh, saya beri label "contoh". Aplikasinya bisa dicoba langsung di https://labtrack-api-b26757zszq-et.a.run.app/app/ dan dokumentasi API-nya di https://labtrack-api-b26757zszq-et.a.run.app/docs.

Tabel 1. Hasil utama pengujian di Google Cloud

Aspek	Hasil	Keterangan
Infrastruktur	62 sumber daya dibuat oleh `terraform apply`	Cloud Run, Cloud SQL, Cloud Storage, Secret Manager, Monitoring, VM operasi
Pipeline CI/CD	4 job lulus (test, test-postgres, terraform, deploy)	Autentikasi ke GCP memakai Workload Identity Federation, tanpa kunci JSON
Uji integrasi	9 dari 9 skenario lulus	Login, data mahasiswa, kehadiran, unggah laporan, URL bertanda tangan
Uji beban 50 pengguna, 5 menit	7.049 permintaan, 0 gagal, p95 69 ms	Target p95 di bawah 800 ms
Uji stress	Jenuh di antara 150 dan 200 pengguna	Hambatan di batas 3 instans Cloud Run, CPU Cloud SQL maksimal 37,5%
Simulasi insiden	Pulih dalam sekitar 20 menit	11 menit di antaranya menunggu Cloud SQL menyala
Uji pemulihan cadangan	Lulus	`pg_dump` 27.521 byte dipulihkan ke basis data sementara dan jumlah barisnya cocok
Ketersediaan berbasis permintaan	99,857%	Target 99,5% tercapai

Hasil yang tidak memenuhi target juga saya tulis apa adanya. Latensi p95 hari itu tercatat 4.928 ms, jauh di atas target 800 ms, karena pada hari yang sama saya menjalankan uji stress sampai 300 pengguna dan simulasi insiden. Keduanya adalah kegiatan terjadwal, dan pembahasannya ada di Bab 6 dan Bab 7.

1. Pendahuluan

1.1 Latar belakang

Laboratorium komputer di kampus menjalankan belasan sesi praktikum per minggu. Untuk setiap sesi, asisten harus mencatat siapa yang hadir, siapa yang terlambat, dan siapa yang izin. Di akhir modul, mahasiswa mengumpulkan laporan dalam bentuk PDF yang kemudian dinilai. Masalahnya muncul saat data itu perlu digabung. Koordinator ingin tahu mahasiswa mana yang kehadirannya sudah di bawah 75% sebelum minggu ujian, tetapi data kehadiran ada di beberapa berkas milik asisten yang berbeda, dengan format yang tidak selalu sama.

Kebutuhannya sebenarnya sederhana. Data harus ada di satu tempat, bisa diakses dari mana saja oleh staf yang berhak, dan tidak boleh hilang. Aplikasi kecil seperti ini cocok dijalankan di cloud karena bebannya naik turun: ramai di jam praktikum dan nyaris kosong di malam hari dan saat libur semester. Membeli server sendiri untuk beban seperti itu berarti membayar mesin yang lebih banyak menganggur.

1.2 Rumusan masalah

1. Bagaimana merancang layanan pencatatan praktikum yang bisa diakses staf laboratorium dari peramban maupun lewat API?
2. Bagaimana memasang layanan tersebut di Google Cloud dengan biaya rendah, tanpa menyimpan rahasia di kode, dan dengan infrastruktur yang bisa dibangun ulang?
3. Bagaimana membuktikan bahwa layanan memenuhi target ketersediaan dan latensi, dan apa yang terjadi ketika komponennya gagal?

1.3 Tujuan

Tujuan teknisnya adalah menghasilkan layanan LabTrack yang berjalan di Cloud Run dengan basis data Cloud SQL, penyimpanan berkas di Cloud Storage, dan pemantauan berbasis SLO. Tujuan lainnya berkaitan dengan asesmen: setiap unit kompetensi yang diujikan harus punya bukti yang bisa ditunjukkan langsung, baik berupa kode, keluaran perintah, maupun tangkapan layar dari Google Cloud.

1.4 Ruang lingkup

Yang termasuk dalam proyek ini:

- API REST untuk akun staf, mahasiswa, mata kuliah, sesi, kehadiran, laporan PDF, dan penilaian.
- Antarmuka web untuk asisten dan koordinator di jalur `/app/`, dilayani oleh layanan Cloud Run yang sama.

- Infrastruktur sebagai kode (Terraform) untuk seluruh sumber daya Google Cloud.
- Pipeline CI/CD di GitHub Actions, skrip operasi harian, laporan SLA otomatis, dan prosedur dukungan teknis.

Yang sengaja tidak dikerjakan: akun untuk mahasiswa (mahasiswa tidak masuk ke sistem; laporannya diunggah oleh asisten), integrasi dengan sistem akademik kampus, dan notifikasi surel ke mahasiswa.

1.5 Pemetaan ke unit kompetensi

Skema Cloud Computing Engineer menurut SKKNI 456/2015 berisi beberapa unit. Enam unit yang diujikan dan bukti utamanya ada di tabel berikut. Rincian per elemen dan kriteria unjuk kerja ada di berkas `docs/00-PEMETAAN-UNIT-KOMPETENSI.md` di repositori.

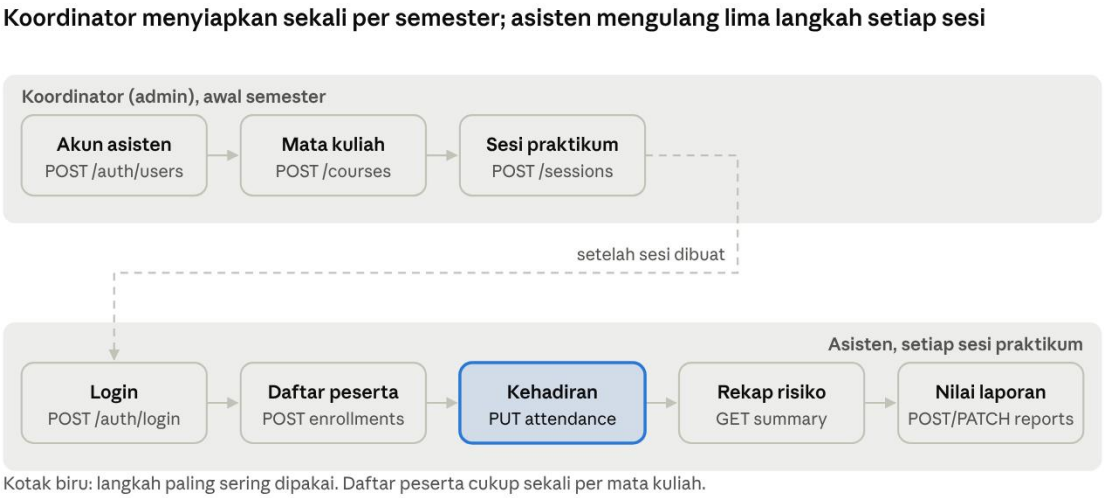
Tabel 2. Unit kompetensi dan bukti utama di proyek LabTrack

Kode unit	Judul unit	Bukti utama
J.631120.001.01	Mendefinisikan terminologi dan konsep cloud computing	Analisis model layanan dan model deployment LabTrack, perbandingan IaaS, PaaS, SaaS pada sistem yang sama
J.631120.002.01	Mengidentifikasi teknologi cloud computing	Pemilihan Cloud Run, Cloud SQL, Cloud Storage, Secret Manager beserta alasannya, Terraform, CI/CD
J.631120.003.01	Mengidentifikasi jenis perangkat keras yang dibutuhkan	Perhitungan kapasitas, uji beban dan stress, perbandingan biaya awan dengan server sendiri
J.631120.013.01	Menjalankan aktivitas rutin pada IaaS, PaaS, dan SaaS	VM operasi, cadangan malam, uji pemulihan, patch, rotasi rahasia, rollback, penskalaan
J.631120.014.01	Memantau sarana dan prasarana sesuai SLA	Dua SLO, enam kebijakan alert, uptime check, dasbor, laporan SLA otomatis
J.631120.015.01	Memberikan dukungan teknis dan report next layer	SOP dukungan berjenjang, `request_id` di setiap galat, simulasi insiden dan laporan pascainsiden

2. Analisis kebutuhan

2.1 Pengguna sistem

Ada dua peran di LabTrack. Koordinator (peran `admin`) menyiapkan data di awal semester: membuat akun asisten, mata kuliah, dan jadwal sesi. Asisten (peran `assistant`) mengerjakan rutinitas per sesi, yaitu mendaftarkan peserta, mengisi kehadiran, mengunggah laporan, dan memberi nilai. Mahasiswa tidak punya akun. Keputusan ini mengurangi permukaan serangan dan menghindari urusan reset kata sandi untuk ratusan orang.



Gambar 1. Alur kerja koordinator di awal semester dan asisten di setiap sesi

2.2 Kebutuhan fungsional

Tabel 3. Kebutuhan fungsional

Kode	Kebutuhan	Peran
F-01	Masuk dengan email dan kata sandi, menerima token JWT	Semua
F-02	Membuat dan melihat akun staf	Koordinator
F-03	Menambah, mencari (NIM atau nama), mengubah, dan menghapus data mahasiswa	Asisten, hapus hanya koordinator
F-04	Membuat mata kuliah dan sesi praktikum	Koordinator
F-05	Mendaftarkan mahasiswa ke mata kuliah, aman bila diulang	Asisten
F-06	Mengisi kehadiran satu sesi sekaligus (hadir, terlambat, izin, alpa)	Asisten
F-07	Melihat rekap kehadiran dan menandai mahasiswa di bawah ambang	Semua
F-08	Mengunggah laporan PDF maksimal 10 MB	Asisten
F-09	Mengunduh laporan lewat tautan yang kedaluwarsa dalam 15 menit	Asisten
F-10	Memberi nilai 0 sampai 100 dan umpan balik	Asisten

2.3 Kebutuhan nonfungsional

Tabel 4. Kebutuhan nonfungsional dan cara pembuktiannya

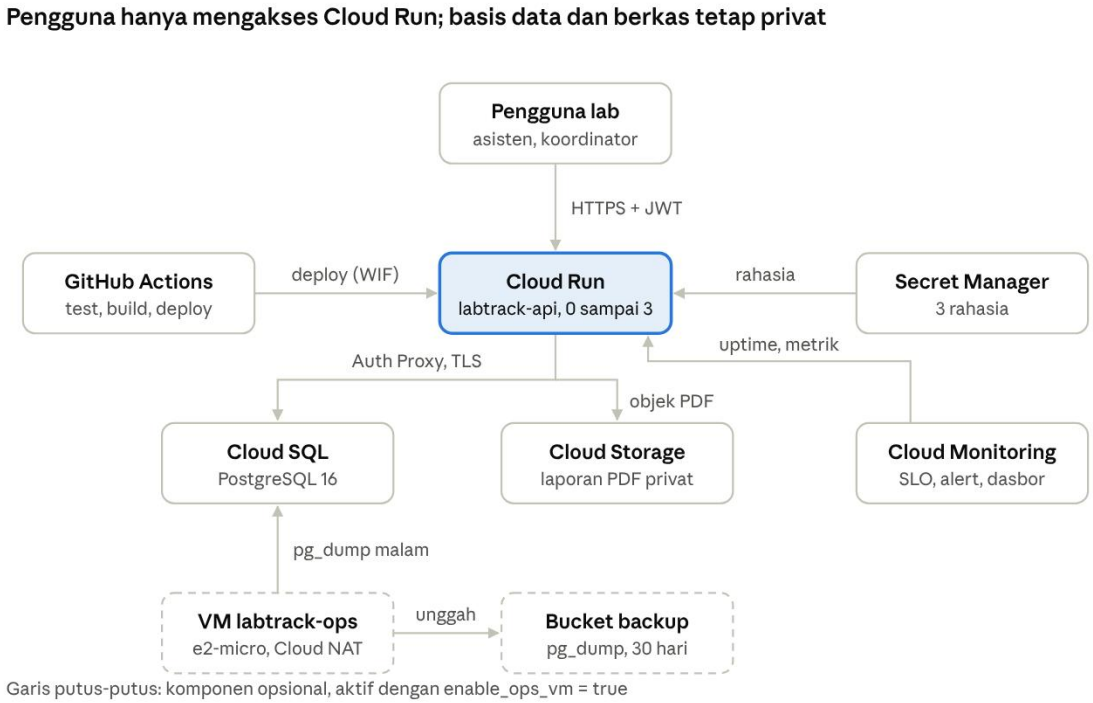
Aspek	Target	Cara membuktikan
Ketersediaan	99,5% permintaan bukan galat 5xx per 30 hari	SLO di Cloud Monitoring dan `scripts/sla_report.py`
Latensi	p95 di bawah 800 ms	SLO latensi, uji beban Locust
Kapasitas	50 pengguna serentak tanpa galat	Uji beban 5 menit
Keamanan	Tanpa rahasia di kode, basis data dan bucket tidak publik	Secret Manager, IAM, Auth Proxy, pemeriksaan konfigurasi
Pemulihan	Cadangan harian dan prosedur pemulihan yang teruji	`pg_dump` malam ke bucket khusus, uji pemulihan
Biaya	Di bawah Rp200.000 per bulan	Anggaran dan alert penagihan, komponen berbayar opsional dimatikan secara bawaan

Angka 99,5% dipilih dengan sadar. Modul pelatihan memakai contoh 99,9%, tetapi untuk layanan kampus dengan satu instans basis data tanpa replika, 99,9% berarti hanya boleh henti 43 menit dalam sebulan. Satu kali pemeliharaan Cloud SQL saja bisa menghabiskannya. Target 99,5% memberi batas sekitar 216 menit per bulan, yang masih wajar untuk aplikasi internal laboratorium dan bisa dipenuhi tanpa replika berbayar.

3. Perancangan sistem

3.1 Arsitektur

Pengguna hanya berhubungan dengan satu titik masuk, yaitu layanan Cloud Run `labtrack-api`. Layanan ini melayani API sekaligus berkas antarmuka web. Basis data Cloud SQL dan bucket laporan tidak bisa diakses langsung dari internet.



Gambar 2. Arsitektur LabTrack di Google Cloud

Alur permintaan berjalan seperti ini. Peramban membuka `/app/`, pengguna masuk, dan antarmuka menyimpan token JWT di `sessionStorage` tab tersebut. Setiap aksi di antarmuka memanggil API yang sama dengan header `Authorization: Bearer`. Cloud Run meneruskan permintaan ke salah satu dari maksimal tiga instans kontainer. Kontainer membuka koneksi ke Cloud SQL lewat konektor Auth Proxy yang dienkripsi TLS dan diotorisasi dengan IAM. Laporan PDF disimpan di bucket privat, dan unduhan diberikan lewat URL bertanda tangan yang berlaku 15 menit.

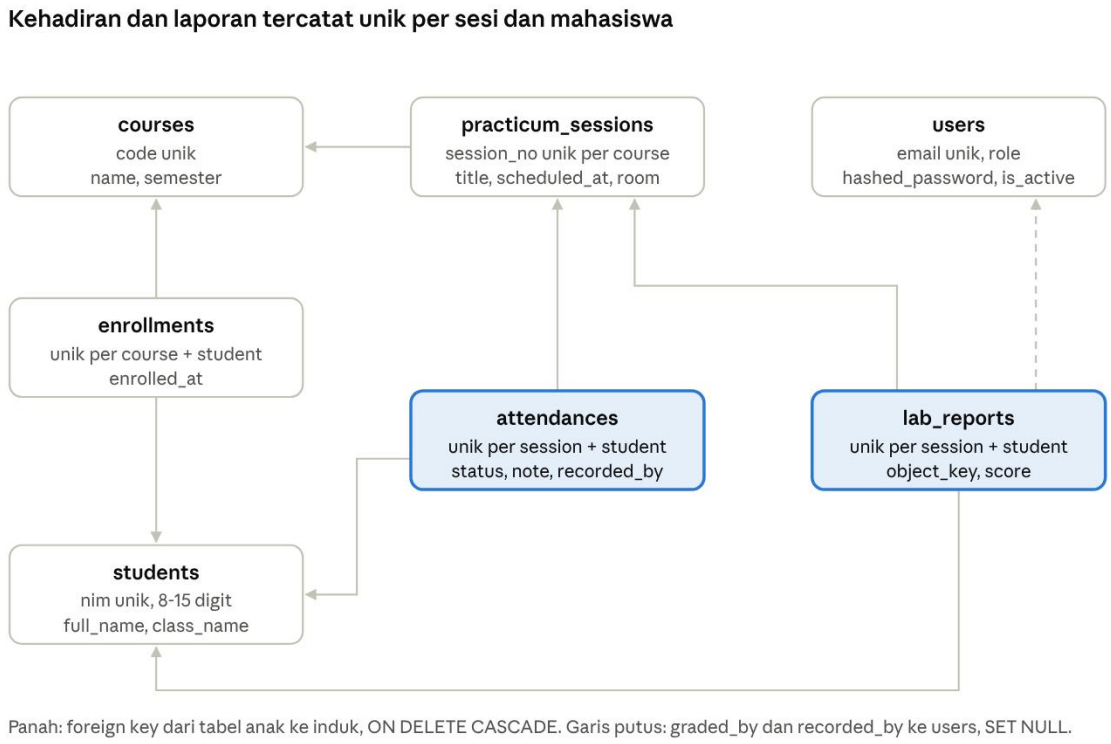
3.2 Pemilihan layanan Google Cloud

Tabel 5. Layanan yang dipakai dan alasannya

Kebutuhan	Layanan	Model	Alasan memilih
Menjalankan API dan web	Cloud Run	PaaS (serverless container)	Bisa turun ke nol instans saat sepi, ditagih per detik pemakaian, HTTPS otomatis
Basis data relasional	Cloud SQL PostgreSQL 16	PaaS (DBaaS)	Cadangan otomatis dan patch dikelola Google; kelas `db-f1-micro` cukup untuk beban ini
Berkas laporan	Cloud Storage	PaaS (object storage)	Murah per GB, akses lewat IAM, mendukung URL bertanda tangan
Rahasia	Secret Manager	PaaS	Kata sandi basis data, kunci JWT, dan kata sandi admin tidak pernah masuk repositori
Citra kontainer	Artifact Registry	PaaS	Satu region dengan Cloud Run, akses lewat IAM
Mesin operasi	Compute Engine e2-micro	IaaS	Bukti aktivitas rutin IaaS: patch OS, snapshot, cadangan terjadwal
Pemantauan	Cloud Monitoring dan Logging	SaaS	SLO, alert, uptime check, dan log terstruktur tanpa memasang server pemantau

Saya sempat menimbang GKE dan App Engine. GKE terlalu berat untuk satu layanan: ada biaya manajemen klaster dan butuh node yang menyala terus. App Engine Standard bisa dipakai, tetapi Cloud Run memberi kebebasan memakai Dockerfile biasa sehingga lingkungan lokal (docker compose) dan produksi menjalankan kontainer yang sama.

3.3 Model data



Gambar 3. Relasi tabel LabTrack

Ada tujuh tabel. Dua aturan paling penting dijaga di level basis data, bukan hanya di kode: satu mahasiswa hanya punya satu catatan kehadiran per sesi, dan satu laporan per sesi. Karena itu mengirim ulang formulir kehadiran tidak membuat data ganda; aplikasi melakukan *upsert*. Rekap kehadiran dihitung dengan rumus (hadir + terlambat) dibagi jumlah sesi yang sudah berlangsung. Sesi dianggap sudah berlangsung begitu ada satu catatan kehadiran di dalamnya, dan mahasiswa yang tidak punya catatan pada sesi itu dihitung tidak hadir.

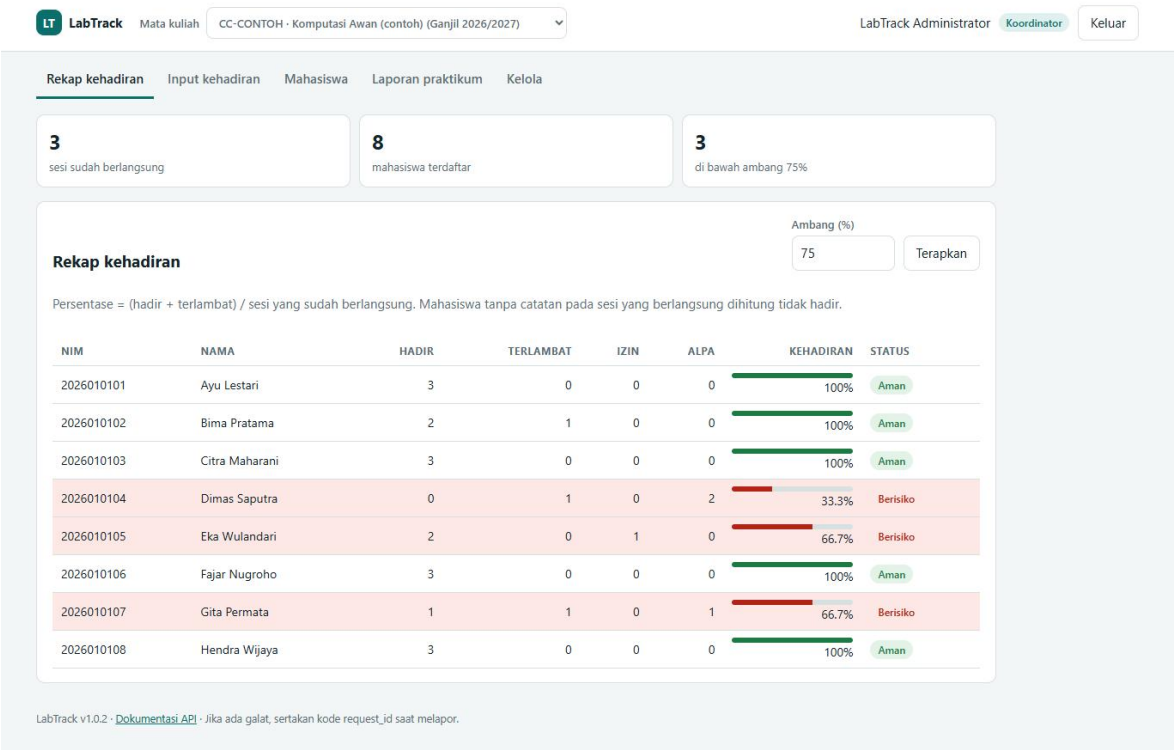
3.4 Keamanan

Beberapa keputusan keamanan yang diambil:

- Kata sandi disimpan sebagai hash bcrypt. Saat email tidak terdaftar, aplikasi tetap membandingkan dengan hash tiruan supaya waktu respons tidak membocorkan email mana yang terdaftar.
- Token JWT berlaku 120 menit. Toleransi selisih jam 30 detik ditambahkan setelah ditemukan galat 401 acak antarinstans (dibahas di Bab 10).
- Berkas yang diunggah diperiksa dari tanda tangan isinya (`%PDF-`), bukan dari jenis konten yang dikirim peramban.
- Akun layanan Cloud Run hanya diberi peran yang dibutuhkan: klien Cloud SQL, pengakses tiga rahasia tertentu, dan pengelola objek di satu bucket.
- GitHub Actions masuk ke Google Cloud lewat Workload Identity Federation. Tidak ada berkas kunci akun layanan yang dibuat atau disimpan.
- VM operasi tidak punya IP publik. Akses SSH hanya lewat IAP, dan akun layanannya hanya bisa menulis objek baru ke bucket cadangan, tidak bisa membaca atau menghapus.

3.5 Antarmuka web

Versi awal LabTrack hanya punya API dan halaman Swagger. Untuk staf laboratorium itu kurang praktis, jadi pada versi 1.0.2 saya menambahkan antarmuka web di `/app/`. Antarmuka ini berupa HTML, CSS, dan JavaScript statis tanpa *framework*, dilayani oleh kontainer yang sama. Tidak ada layanan tambahan, tidak ada biaya tambahan, dan aturan otorisasinya tetap ditegakkan di API. Pengunjung yang membuka alamat utama dengan peramban otomatis diarahkan ke `/app/`, sedangkan klien API tetap menerima JSON.



Gambar 4. Rekap kehadiran di antarmuka web, data contoh mata kuliah CC-CONTOH

Setiap pesan galat di antarmuka menampilkan "Kode" berupa `request_id`. Kode ini sama dengan yang tercatat di Cloud Logging, sehingga staf yang melapor cukup menyalin kode itu dan petugas dukungan bisa langsung menemukan log permintaannya.

4. Implementasi

4.1 Struktur repositori

Tabel 6. Struktur repositori

Folder	Isi
`app/`	Kode FastAPI: model SQLAlchemy, skema Pydantic, router, keamanan, penyimpanan, berkas web di `app/static/`
`tests/`	72 pengujian pytest, cakupan kode 95%
`infra/terraform/`	Seluruh sumber daya Google Cloud, termasuk SLO, alert, dasbor, dan VM operasi
`.github/workflows/`	Pipeline CI/CD empat job
`scripts/ops/`	13 skrip operasi: deploy, rollback, skala, rotasi rahasia, cadangan, pemulihan, VM
`scripts/loadtest/`	Skenario Locust untuk uji beban dan stress
`scripts/evidence/`	Skrip pengambil tangkapan layar dan grafik dari Monitoring API
`docs/`	Dokumen kompetensi 00 sampai 07, panduan deployment, laporan SLA, folder bukti

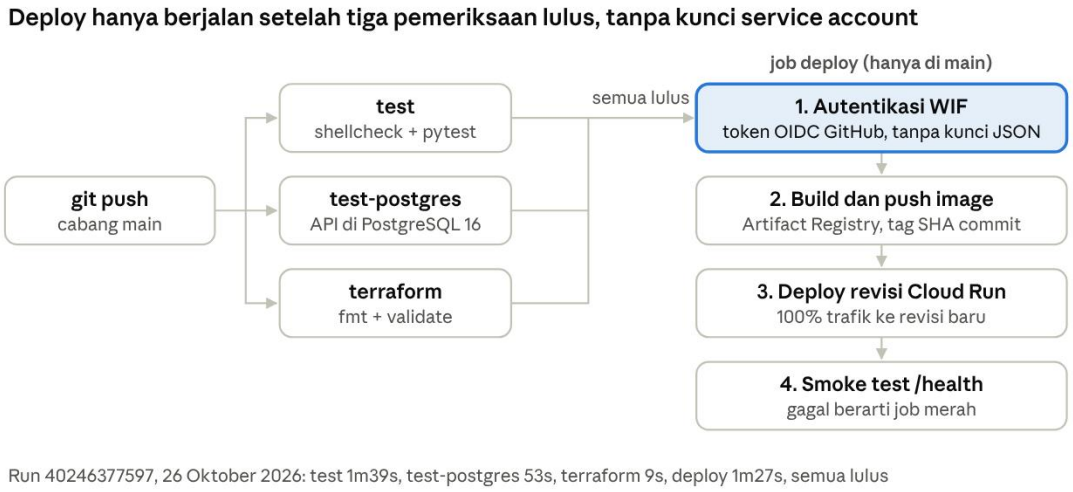
4.2 Infrastruktur sebagai kode

Semua sumber daya dibuat oleh Terraform 1.9.8 dengan provider google 6.50.0. Satu kali `terraform apply` di proyek kosong membuat 62 sumber daya. Komponen yang berbayar dan tidak wajib berada di balik variabel boolean yang bernilai `false` secara bawaan, dengan perkiraan biaya bulanan ditulis di deskripsi variabelnya. Dua yang saya nyalakan untuk pembuktian adalah `enable_sla_alerts` (kebijakan alert SLO) dan `enable_ops_vm` (VM operasi beserta Cloud NAT dan jadwal snapshot).

```
# infra/terraform/terraform.tfvars (nilai yang dipakai, rahasia tidak disimpan di sini)
project_id        = "labtrack-1026146"
region            = "asia-southeast2"
enable_sla_alerts = true
enable_ops_vm     = true
ops_vm_region     = "us-central1"   # e2-micro masuk free tier di region ini
```

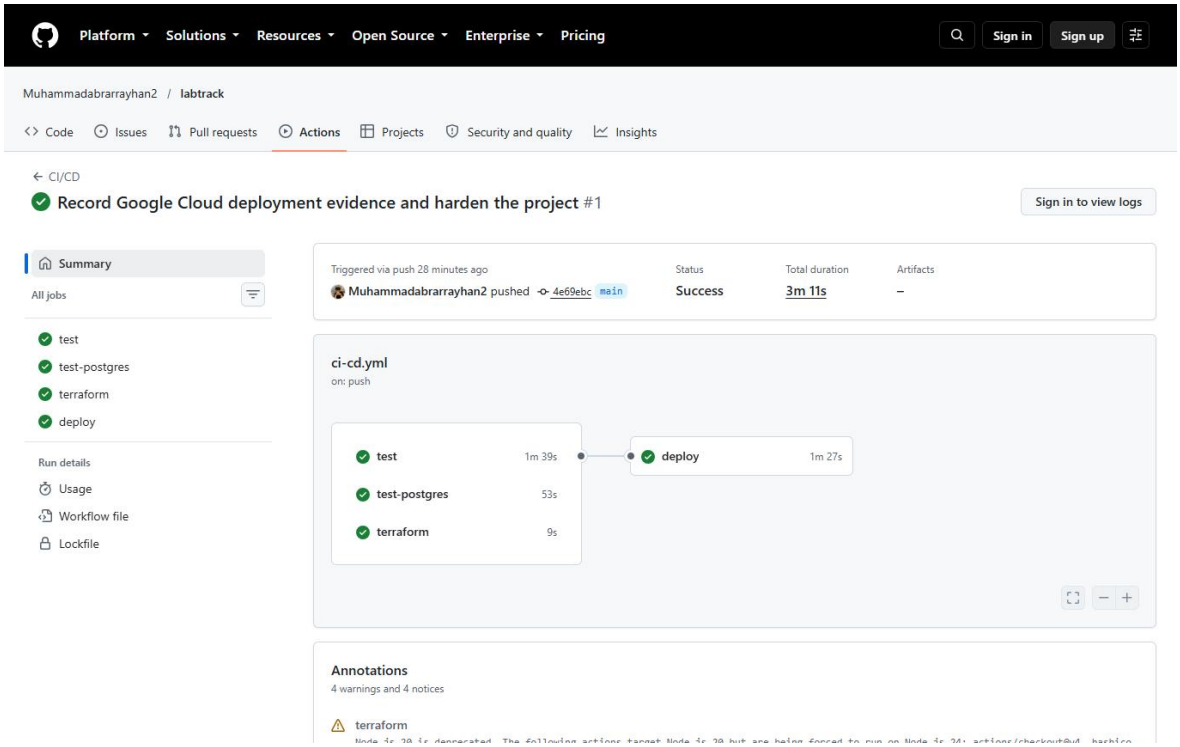
VM operasi sengaja ditempatkan di `us-central1`. Mesin e2-micro di region itu masuk kuota gratis Google Cloud, sedangkan di Jakarta tidak. Untuk tugas cadangan malam, selisih latensi lintas region tidak berpengaruh.

4.3 Pipeline CI/CD



Gambar 5. Pipeline GitHub Actions dari push sampai smoke test

Setiap *push* ke cabang `main` menjalankan tiga pemeriksaan paralel. Job `test` menjalankan shellcheck pada skrip operasi dan pytest dengan SQLite. Job `test-postgres` menjalankan pytest yang sama terhadap PostgreSQL 16 sungguhan, karena beberapa perilaku (misalnya kunci unik dan zona waktu) berbeda antara SQLite dan PostgreSQL. Job `terraform` menjalankan `fmt` dan `validate`. Hanya jika ketiganya lulus, job `deploy` membangun citra, memberinya tag SHA commit, mendorongnya ke Artifact Registry, memasang revisi baru di Cloud Run, lalu memanggil `/health`. Bila pemeriksaan kesehatan gagal, job ditandai merah.



Gambar 6. Run GitHub Actions dengan keempat job lulus

4.4 Pengujian otomatis

Tabel 7. Kelompok pengujian

Berkas	Yang diuji
`test_api.py`	Alur utama API: login, CRUD mahasiswa, kehadiran, rekap, unggah dan nilai laporan, otorisasi peran
`test_request_id.py`	Setiap galat 4xx dan 5xx membawa `request_id` di badan respons dan header
`test_security.py`	Toleransi selisih jam JWT, token kedaluwarsa, token dengan tanda tangan salah
`test_web.py`	Antarmuka web dilayani di `/app/`, pengalihan dari `/` untuk peramban
`test_sla_report.py`	Perhitungan ketersediaan, anggaran galat, jendela uptime
`test_ops_scripts.py`	Skrip operasi menolak argumen salah dan mencetak rencana sebelum mengubah apa pun
`test_loadtest_stages.py`, `test_integration_check.py`	Profil beban Locust dan skrip uji integrasi

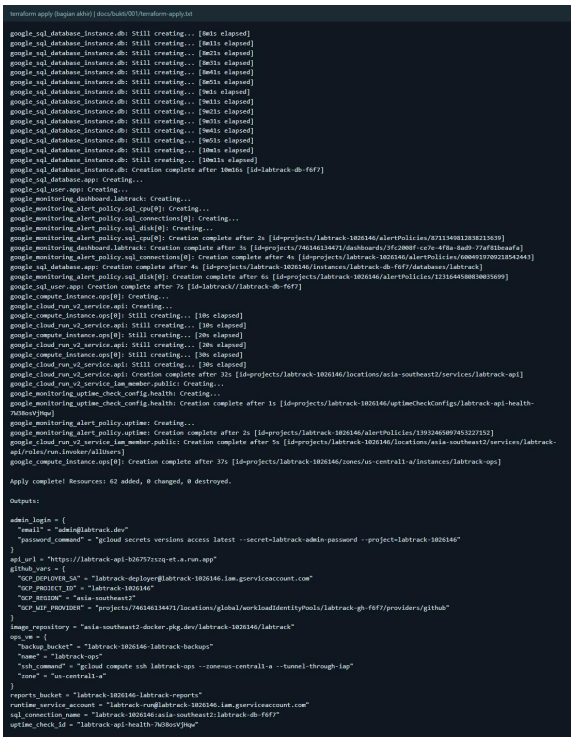
Hasil terakhir: 72 pengujian lulus dengan cakupan 95% untuk paket `app`.

5. Deployment di Google Cloud

5.1 Langkah deployment

Deployment pertama dilakukan dari laptop, setelah itu pembaruan berjalan lewat pipeline. Urutannya:

1. Membuat proyek `labtrack-1026146`, menautkan akun penagihan, dan memasang anggaran Rp200.000 per bulan dengan notifikasi surel.
2. Mengaktifkan API yang dibutuhkan, lalu membangun citra pertama dengan Cloud Build dan menyimpannya di Artifact Registry.
3. Menjalankan `terraform init`, `plan`, dan `apply`. Rencana menunjukkan 62 sumber daya baru tanpa perubahan atau penghapusan.
4. Menjalankan `scripts/api_integration_check.py` terhadap URL Cloud Run. Sembilan skenario lulus.
5. Mengatur proteksi cabang `main` di GitHub sehingga perubahan hanya bisa masuk bila tiga pemeriksaan wajib lulus.



Gambar 7. Keluaran terraform apply

5.2 Hasil uji integrasi

Skrip uji integrasi berjalan dari luar Google Cloud, persis seperti klien sungguhan. Ia masuk sebagai admin, membuat mahasiswa dan mata kuliah sementara, mengisi kehadiran, mengunggah PDF kecil, meminta URL unduhan, mengunduh berkasnya, memberi nilai, lalu membersihkan data yang dibuat.

scripts/api_integration_check.py | docs/bukti/002/integration-report.md

```
# Laporan Uji Integrasi API LabTrack

- Target: `https://labtrack-api-b26757zszq-et.a.run.app`
- Waktu: 2026-10-26 01:10:06 WIB
- Dihasilkan oleh: `scripts/api_integration_check.py`
- Hasil: **9 dari 9 pemeriksaan lulus**

| No | Pemeriksaan | Permintaan | Diharapkan | Aktual | Latensi (ms) | request_id | Status |
|---|---|---|---|---|---|---|---|
| 1 | Health and database | `GET /health` | 200 | 200, database=ok | 344.0 | `70a6243f939e0e9edc5adad199e2a6d3` | LULUS |
| 2 | Login admin | `POST /auth/login` | 200 | 200 | 372.2 | `24e553fa81f9e64cdc5adad199e2a69e` | LULUS |
| 3 | Create student | `POST /students` | 201 | 201 | 108.5 | `cbb33c1aebc87ff2dc5adad199e2ae7c` | LULUS |
| 4 | Read back from database | `GET /students?q=991792951805` | 200 | 200, total=1 | 44.4 | `9b79eb6d8a0b7a15dc5adad199e2af60` | LULUS |
| 5 | Update student | `PATCH /students/1` | 200 | 200 | 48.5 | `76bd140a9dd88149dc5adad199e2a961` | LULUS |
| 6 | Duplicate NIM rejected with request_id | `POST /students` | 409 | 409 | 50.2 | `c683667eb879aceedc5adad199e2a362` | LULUS |
| 7 | No token rejected with request_id | `GET /students` | 401 | 401 | 20.5 | `3ae88a4cc9106448dc5adad199e2ac33` | LULUS |
| 8 | Delete test student | `DELETE /students/1` | 204 | 204 | 59.8 | `8244d17931442632dc5adad199e2a550` | LULUS |
| 9 | Deleted student is gone | `GET /students/1` | 404 | 404 | 41.4 | `e9b4e2277de8a404dc5adad199e2a680` | LULUS |
```

Gambar 8. Laporan uji integrasi, 9 dari 9 lulus

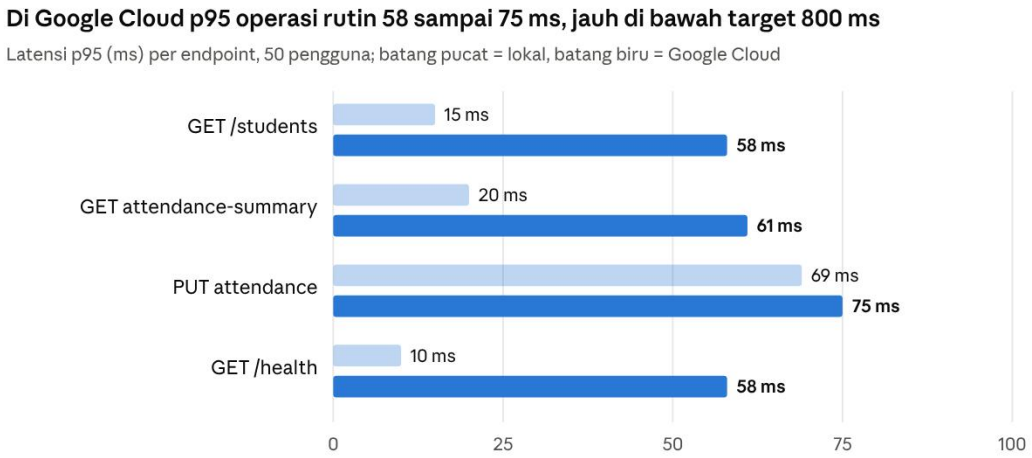
6. Pengujian kinerja

6.1 Uji beban

Uji beban memakai Locust dengan 50 pengguna virtual selama 5 menit. Campuran permintaannya meniru pola nyata: lebih banyak membaca daftar mahasiswa dan rekap daripada menulis kehadiran.

Tabel 8. Hasil uji beban 50 pengguna di Google Cloud

Besaran	Nilai
Total permintaan	7.049
Permintaan gagal	0
Throughput rata-rata	23,58 permintaan per detik
p95 seluruh endpoint	69 ms
Target p95	800 ms



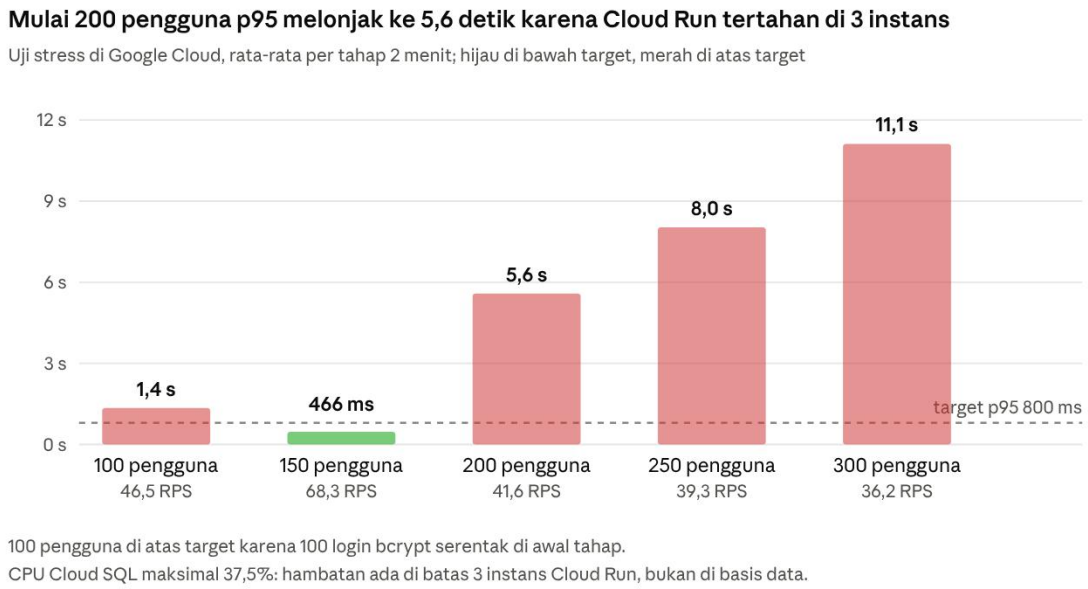
Gambar 9. Perbandingan p95 per endpoint di lingkungan lokal dan Google Cloud

Latensi di Google Cloud lebih tinggi daripada di laptop, terutama untuk endpoint yang ringan. Penyebab utamanya jarak jaringan dari laptop penguji ke region Jakarta dan

lompatan tambahan lewat Auth Proxy ke Cloud SQL. Selisih itu tidak berarti apa-apa dibanding target 800 ms.

6.2 Uji stress

Uji stress menaikkan beban bertahap dari 100 sampai 300 pengguna, masing-masing 2 menit, untuk mencari titik jenuh.

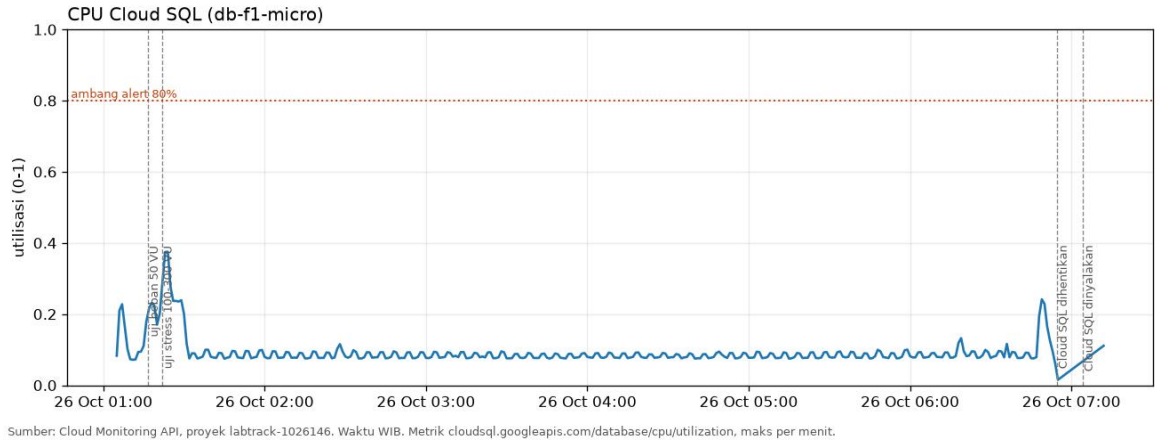


Gambar 10. Latensi p95 per tahap uji stress

Sistem masih memenuhi target pada 150 pengguna dengan p95 466 ms dan 68,3 permintaan per detik. Mulai 200 pengguna p95 melonjak ke 5,6 detik, dan throughput justru turun. CPU Cloud SQL hanya mencapai 37,5%, jadi basis data bukan penyebabnya. Hambatannya ada di batas maksimal tiga instans Cloud Run yang saya tetapkan untuk menjaga biaya dan jumlah koneksi basis data. Koneksi Cloud SQL sempat mencapai 24 dari 25 yang tersedia. Galat yang muncul hanya 0,08%, semuanya HTTP 429 dari Cloud Run ketika antrean penuh.

Tahap 100 pengguna terlihat di atas target, dan itu bukan kesalahan pengukuran. Di awal tahap, 100 pengguna login hampir bersamaan, dan setiap login menjalankan bcrypt yang memang sengaja dibuat lambat. Setelah fase login lewat, latensi kembali normal.

Pemulihan setelah stress tidak sempat terekam dalam satu sesi Locust karena riwayatnya berhenti di detik ke-579. Pengukuran pemulihan saya lakukan terpisah sekitar lima jam kemudian dari kondisi diam, dan hasilnya kembali normal. Catatan ini saya tulis supaya tidak ada kesan bahwa kedua pengukuran berasal dari satu rangkaian.



Gambar 11. CPU Cloud SQL selama uji stress

6.3 Kesimpulan kapasitas

Untuk laboratorium dengan 3 sampai 4 kelas paralel, beban puncaknya sekitar 40 sampai 60 asisten dan koordinator yang aktif bersamaan. Konfigurasi sekarang menampung 150 pengguna serentak dengan latensi di bawah target, jadi ruang cadangannya sekitar dua kali lipat. Bila suatu saat dibutuhkan lebih, langkah pertama adalah menaikkan batas instans Cloud Run sekaligus kelas Cloud SQL, karena batas koneksi `db-f1-micro` hanya 25.

7. Operasi dan pemantauan

7.1 Aktivitas rutin

Tabel 9. Aktivitas rutin yang sudah dijalankan di Google Cloud

Aktivitas	Lapisan	Skrip	Hasil
Deploy versi baru	PaaS	`deploy-image.sh`	Revisi baru menerima 100% trafik setelah `/health` lulus
Rollback	PaaS	`rollback.sh`	Trafik kembali ke revisi sebelumnya dalam hitungan detik
Penskalaan	PaaS	`scale.sh`	Batas instans diubah tanpa deploy ulang
Rotasi kunci JWT	PaaS	`rotate-secret.sh`	Token lama ditolak 401, login baru berhasil 200
Cadangan basis data	IaaS dan PaaS	Cron di VM, `sql-backup-now.sh`	`pg_dump` 27.521 byte di bucket cadangan
Uji pemulihan	PaaS	`sql-restore-test.sh`	Dipulihkan ke basis data sementara, jumlah baris cocok, sekitar 27 menit
Patch OS	IaaS	`vm-patch.sh`	Lima langkah tercatat di change log
Snapshot disk	IaaS	`vm-snapshot-now.sh` dan jadwal harian	Snapshot dibuat dan dipertahankan 7 hari
Laporan VM	IaaS	`vm-report.sh`	Debian 12, KVM, tanpa IP publik, keluar lewat NAT
Nonaktifkan akun dan reset sandi	SaaS (aplikasi)	Runbook di dokumen 04	Akun nonaktif tidak bisa login, sandi baru berlaku

scripts/ops/sql-restore-test.sh | docs/bukti/013/restore-verification-20261026-0625.md

```
# Laporan Verifikasi Restore Cloud SQL

| Butir | Nilai |
|---|---|
| Waktu uji (WIB) | 2026-10-26 06:53 |
| Proyek | labtrack-1026146 |
| Instans sumber | labtrack-db-f6f7 |
| Backup yang dipulihkan | ID 1792970383825, jendela backup 2026-10-25T23:19:43.825Z |
| Instans uji sementara | labtrack-restore-test-202610252325 (dihapus otomatis setelah uji) |
| Durasi total | sekitar 27 menit, memenuhi RTO 2 jam |
| Hasil | **LULUS** |

## Perbandingan jumlah baris

Salinan hasil restore boleh lebih kecil dari produksi karena data yang ditulis setelah jendela backup belum ada di backup. Salinan tidak boleh kosong
bila produksi berisi data, dan tidak boleh lebih besar dari produksi.

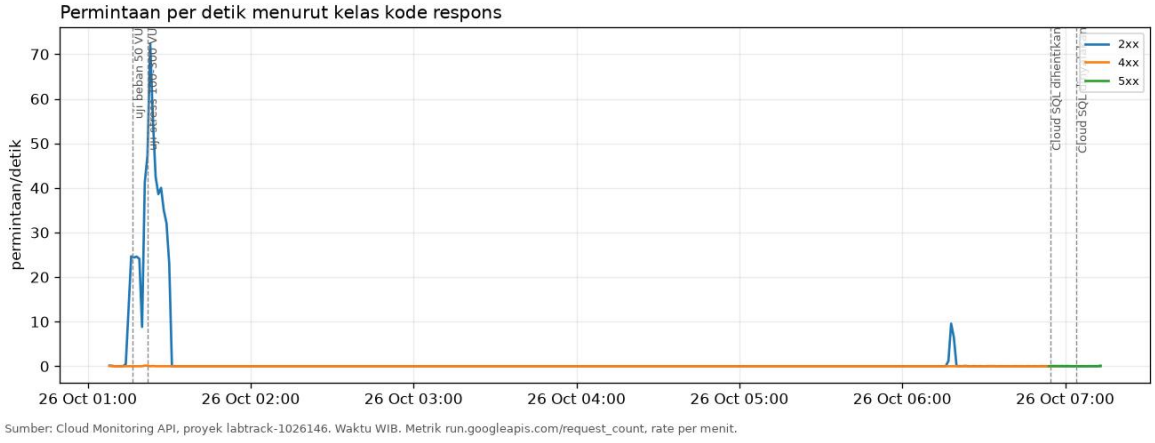
| Tabel | Produksi | Hasil restore | Status |
|---|---|---|---|
| users | 2 | 1 | OK |
| students | 12 | 12 | OK |
| courses | 1 | 1 | OK |
| enrollments | 12 | 12 | OK |
| practicum_sessions | 4 | 4 | OK |
| attendances | 48 | 48 | OK |
| lab_reports | 41 | 41 | OK |

Dihasilkan oleh scripts/ops/sql-restore-test.sh.
```

Gambar 12. Verifikasi uji pemulihan cadangan

7.2 Pemantauan dan SLO

Terraform membuat dua SLO di Cloud Monitoring, satu untuk ketersediaan dan satu untuk latensi, masing-masing dengan jendela 30 hari bergulir. Di atasnya ada enam kebijakan alert: layanan tidak menjawab uptime check, anggaran galat ketersediaan terbakar lebih dari 10 kali laju normal dalam 1 jam, rasio galat 5xx di atas 5% selama 5 menit, CPU Cloud SQL di atas 80% selama 10 menit, disk Cloud SQL di atas 80%, dan koneksi Cloud SQL mendekati batas. Notifikasi dikirim ke surel pemilik proyek. Dasbor berisi delapan widget yang menampilkan permintaan per kelas respons, latensi, instans aktif, CPU dan koneksi Cloud SQL, serta status SLO.



Gambar 13. Permintaan Cloud Run per kelas respons selama hari pengujian

Laporan SLA bulanan dibuat oleh `scripts/sla_report.py`. Skrip ini menarik data dari Monitoring API, menghitung ketersediaan berbasis permintaan dan berbasis waktu, lalu menulis laporan Markdown di `docs/laporan-sla/`.

Tabel 10. Laporan SLA 26 Oktober 2026

Indikator	Target	Realisasi	Status
Ketersediaan (permintaan bukan 5xx)	99,5%	99,857%	Tercapai
Latensi p95	di bawah 800 ms	4.928 ms	Tidak tercapai
Permintaan di bawah 800 ms	95%	61,79%	Tidak tercapai
Ketersediaan berbasis waktu (uptime check)	99,5%	94,521%	Tidak tercapai

Dari 34.342 permintaan hari itu, 49 berakhir dengan galat 5xx, sehingga anggaran galat yang terpakai 28,5%. Angka latensi dan uptime yang gagal berasal dari dua kegiatan yang sengaja saya jalankan: uji stress sampai 300 pengguna dan simulasi insiden. Dalam praktik operasional, kegiatan seperti itu dijadwalkan dan dikecualikan dari perhitungan SLA. Saya tetap menampilkan angka mentahnya karena pengecualian adalah keputusan kebijakan, bukan alasan untuk menyembunyikan data.

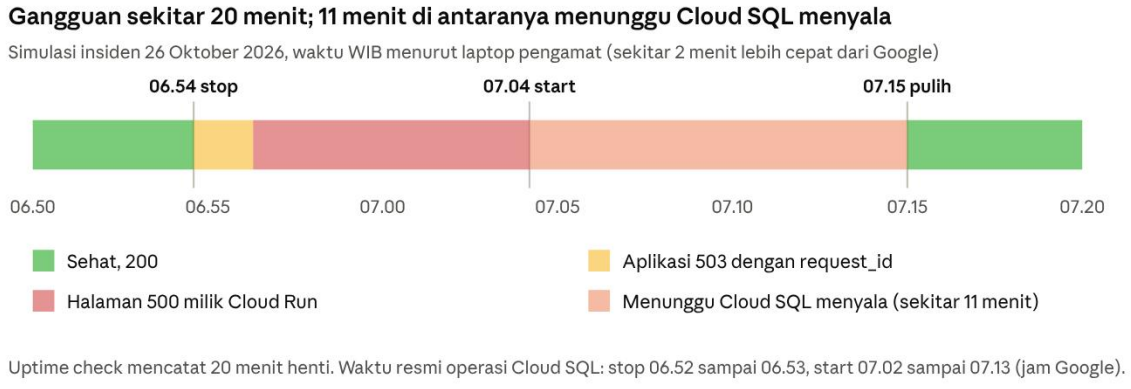
8. Dukungan teknis dan simulasi insiden

8.1 Alur dukungan berjenjang

Dukungan teknis dibagi tiga level. Level 1 adalah asisten senior atau admin laboratorium yang menerima laporan dari pengguna dan menangani masalah akun, kesalahan input, atau cara pakai. Level 2 adalah pengelola LabTrack yang membaca log, memeriksa dasbor, dan menjalankan skrip operasi. Level 3 adalah pihak di luar tim: dukungan Google Cloud untuk masalah platform, atau pengembang bila ditemukan cacat kode. Setiap eskalasi wajib membawa `request_id`, waktu kejadian, dan langkah yang sudah dicoba. Templat isu di GitHub sudah menyediakan kolom-kolom itu.

8.2 Simulasi insiden

Pada 26 Oktober 2026 pukul 06.54 WIB saya menghentikan instans Cloud SQL dengan sengaja untuk melihat apa yang dialami pengguna dan seberapa cepat layanan bisa pulih.

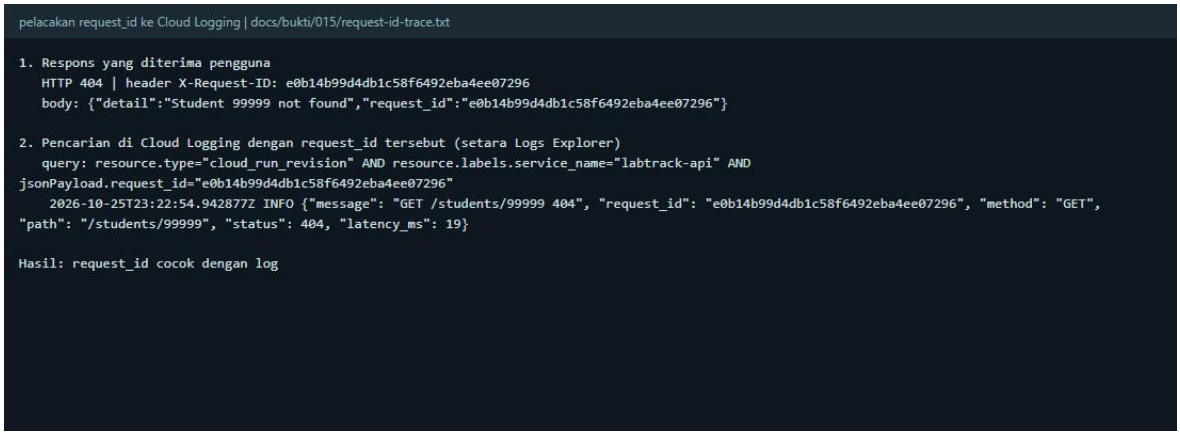


Gambar 14. Linimasa simulasi insiden

Pada menit-menit pertama aplikasi masih hidup dan menjawab 503 dengan `request_id`, sesuai rancangan. Setelah itu muncul masalah yang tidak saya perkirakan: instans baru Cloud Run gagal melewati *startup probe* karena probe memanggil `/health` yang ikut memeriksa basis data. Akibatnya sebagian pengguna menerima halaman 500 bawaan Cloud Run tanpa `request_id`. Menyalakan kembali Cloud SQL ternyata memakan waktu sekitar 11 menit (07.02 sampai 07.13 menurut jam Google), lebih lama dari dugaan saya. Layanan pulih pukul 07.15 WIB menurut jam laptop. Total gangguan sekitar 20 menit.

Tabel 11. Temuan dari simulasi insiden

No.	Temuan	Tindak lanjut
1	Startup probe memeriksa basis data, sehingga instans baru tidak bisa naik saat Cloud SQL mati	Pisahkan probe *liveness* dari pemeriksaan basis data; tercatat di batasan yang diketahui
2	Pengguna melihat halaman galat Cloud Run tanpa `request_id`	SOP level 1 ditambah langkah mencatat waktu kejadian bila kode tidak tersedia
3	Menyalakan Cloud SQL butuh sekitar 11 menit	Jangan mematikan Cloud SQL di jam praktikum; jadwal hemat biaya hanya di malam hari dan akhir pekan
4	Jam laptop berselisih sekitar 2 menit dengan jam Google	Laporan insiden selalu memakai stempel waktu dari Cloud Logging



Gambar 15. Penelusuran satu permintaan di Cloud Logging dengan request_id

9. Biaya

Tabel 12. Perkiraan biaya bulanan di Google Cloud

Komponen	Perkiraan per bulan	Catatan
Cloud SQL `db-f1-micro`, menyala terus	US$10 sampai 15	Biaya terbesar; bisa dimatikan di luar jam pakai
Cloud Run	Mendekati US$0	Turun ke nol instans saat tidak ada permintaan, beban demo masih di kuota gratis
Cloud Storage, Artifact Registry, Secret Manager	Di bawah US$1	Volume kecil
VM operasi e2-micro di us-central1	US$0 untuk mesin, sekitar US$1 sampai 3 untuk NAT dan snapshot	Opsional, `enable_ops_vm`

Komponen	Perkiraan per bulan	Catatan
Cloud Monitoring	US$0	Metrik bawaan dan SLO tidak ditagih
Total	**Sekitar Rp160.000 sampai Rp240.000**	Kurs contoh Rp16.000 per US$

Sebagai pembanding, dokumen 03 menghitung server sendiri di lokasi (contoh harga pasar) dengan biaya awal sekitar Rp30 juta dan biaya bulanan sekitar Rp1,29 juta setelah penyusutan, listrik, dan internet. Untuk beban sekecil LabTrack, model sewa di awan jauh lebih murah. Perbandingan ini bisa berbalik bila bebannya stabil dan tinggi sepanjang hari, tetapi itu bukan kasus laboratorium kampus.

Setelah asesmen, biaya bisa ditekan hampir ke nol dengan `scripts/ops/sql-start-stop.sh stop`, `scripts/ops/vm-start-stop.sh stop`, dan mengubah `enable_sla_alerts` menjadi `false`. Bila proyek tidak lagi dibutuhkan, `terraform destroy` menghapus semuanya.

10. Masalah yang ditemui dan penyelesaiannya

Beberapa masalah baru muncul ketika sistem berjalan di Google Cloud dan tidak terlihat di laptop. Saya mencatatnya karena justru bagian ini yang paling banyak mengajarkan sesuatu.

Tabel 13. Masalah selama deployment dan operasi

Masalah	Penyebab	Penyelesaian
Galat 401 acak setelah login, hanya di Cloud Run	Klaim `iat` token dibuat di satu instans lalu diperiksa di instans lain yang jamnya sedikit tertinggal	Toleransi selisih jam 30 detik di `decode_token`, pengujian baru, rilis 1.0.1
Cadangan dari VM gagal 403	`gcloud storage cp` memerlukan izin baca objek, padahal akun layanan VM sengaja hanya boleh menulis	Unggah lewat JSON API dengan `curl` yang hanya butuh `objects.create`
Cloud Build ditolak saat build pertama	API Compute belum aktif dan akun layanan bawaan belum punya peran builder	Mengaktifkan API dan memberi peran `cloudbuild.builds.builder`
Jaringan `default` terbuat otomatis dengan SSH dan RDP terbuka ke internet	Perilaku bawaan saat API Compute diaktifkan	Jaringan `default` dihapus setelah disetujui; VM memakai VPC sendiri
Skrip uji pemulihan gagal	Opsi `--labels` tidak dikenal di versi gcloud yang terpasang	Opsi dihapus dan skrip diuji ulang

11. Penutup

11.1 Kesimpulan

LabTrack berjalan di Google Cloud dengan arsitektur serverless yang biayanya mengikuti pemakaian. Seluruh infrastrukturnya bisa dibangun ulang dari kode, rilisnya otomatis dan tanpa kunci rahasia yang tersimpan, dan kesehatannya diukur dengan SLO yang jelas.

Pengujian menunjukkan layanan menampung 50 pengguna dengan p95 69 ms tanpa galat, dan masih memenuhi target pada 150 pengguna. Simulasi insiden membuktikan bahwa alur dukungan dan `request_id` bekerja, sekaligus membuka satu kelemahan desain pada probe kesehatan yang belum saya sadari sebelumnya.

11.2 Saran pengembangan

1. Memisahkan endpoint *liveness* dari pemeriksaan basis data supaya instans tetap bisa naik saat Cloud SQL bermasalah.
2. Pindah dari `create_all` ke migrasi Alembic sebelum ada perubahan skema di produksi.
3. Menambahkan endpoint untuk menonaktifkan akun dan mereset kata sandi, supaya tidak perlu lewat Cloud SQL Studio.
4. Memakai IP privat untuk Cloud SQL bila anggaran mengizinkan, dan menaikkan kelas basis data bila jumlah pengguna serentak lebih dari 150.
5. Menghapus berkas PDF di bucket ketika data mahasiswa dihapus.

Lampiran

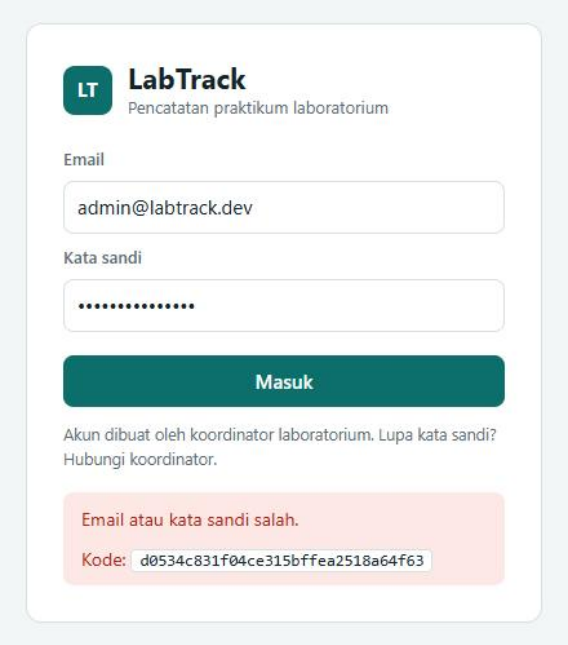
Daftar bukti

Seluruh bukti tersimpan di folder `docs/bukti/` di repositori, dikelompokkan per unit kompetensi. Tangkapan layar ada di `docs/bukti/screenshots/` dengan indeks di berkas `README.md` di folder tersebut.

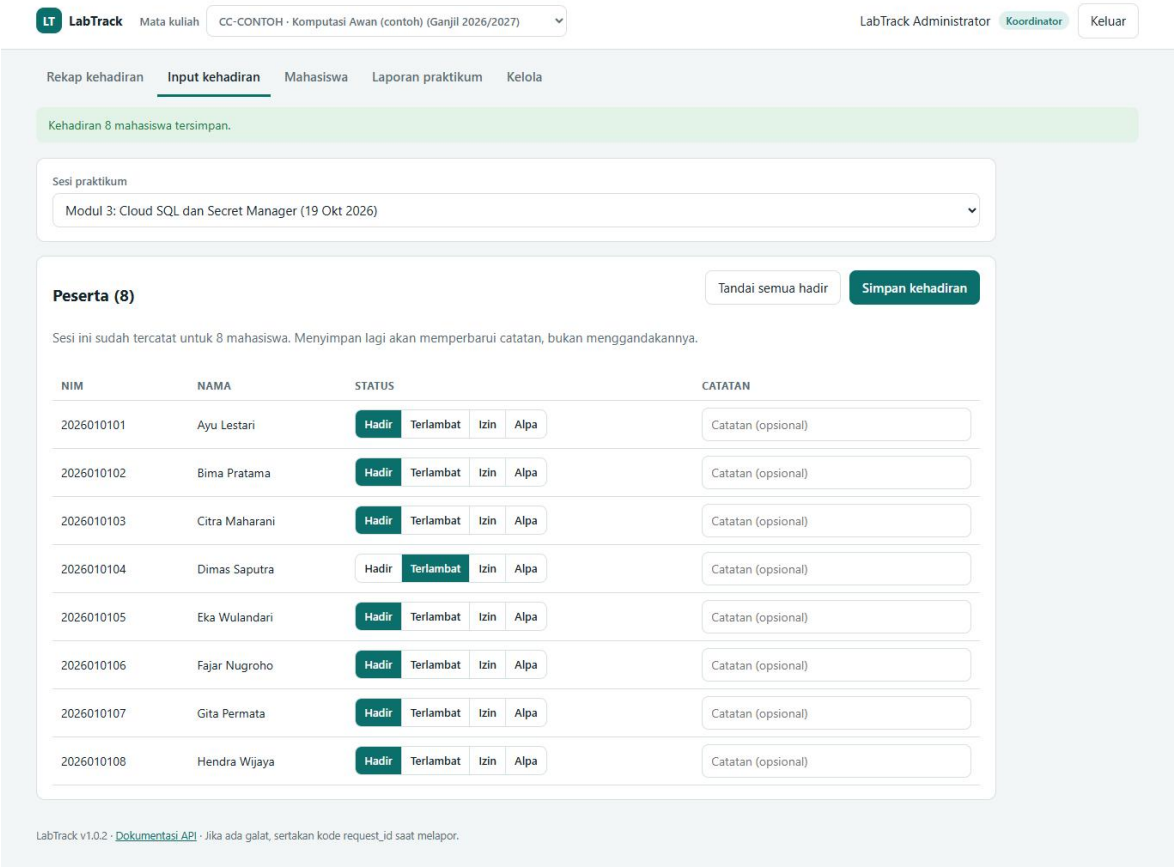
Tabel 14. Bukti per unit kompetensi

Unit	Lokasi bukti	Contoh isi
001	`docs/01-konsep-cloud-computing.md`	Analisis model layanan dan deployment
002	`docs/02-teknologi-cloud.md`, `docs/bukti/002/`	Keluaran terraform apply, run CI/CD, laporan integrasi
003	`docs/03-kebutuhan-perangkat-keras.md`, `docs/bukti/003/`	Hasil Locust, grafik Monitoring
013	`docs/04-aktivitas-rutin-iaas-paas-saas.md`, `docs/bukti/013/`	Log cadangan, change log patch, verifikasi pemulihan
014	`docs/05-sla-dan-pemantauan.md`, `docs/laporan-sla/`	Laporan SLA, uptime check, grafik permintaan
015	`docs/06-sop-dukungan-teknis.md`, `docs/bukti/015/`	Linimasa insiden, penelusuran `request_id`

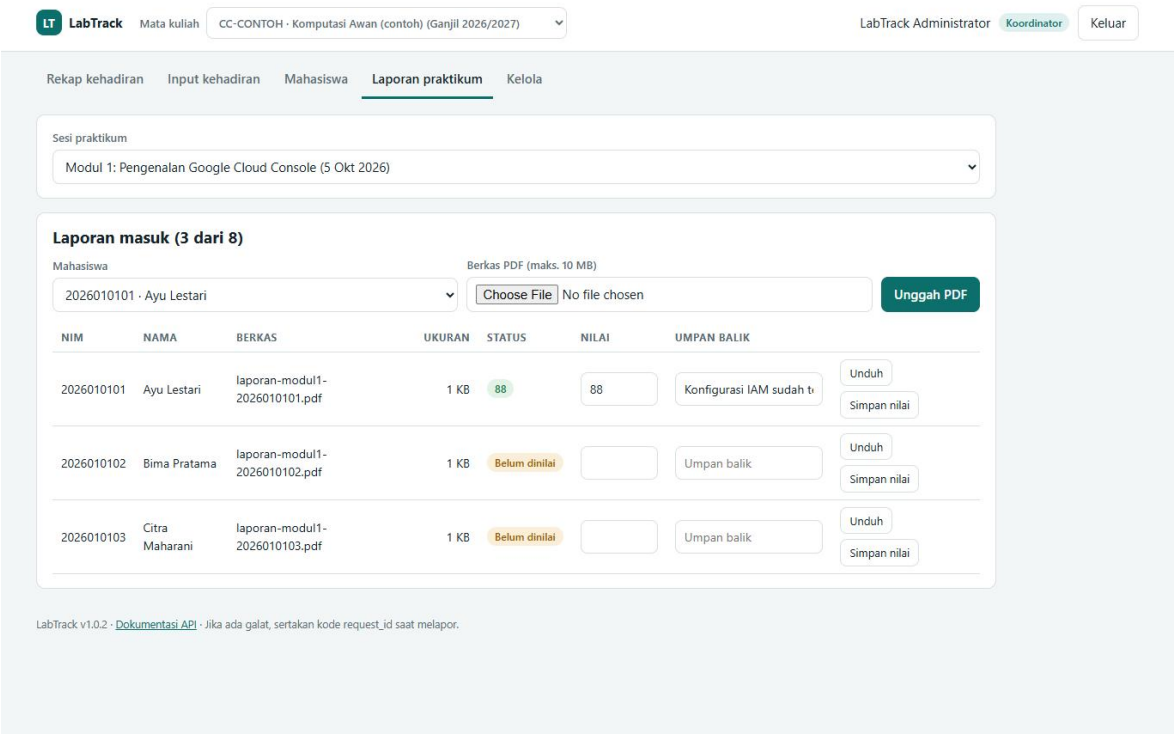
Tangkapan layar antarmuka web



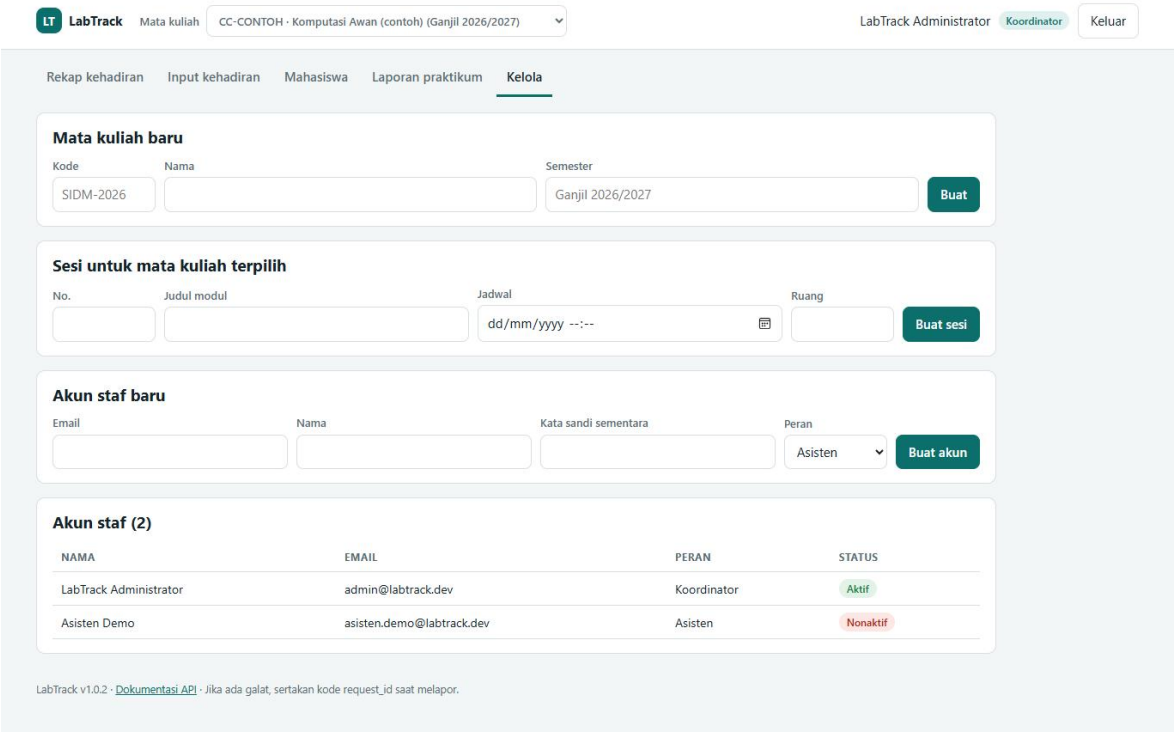
Gambar 16. Halaman masuk; galat menampilkan kode request_id



Gambar 17. Input kehadiran satu sesi sekaligus



Gambar 18. Laporan praktikum: unggah, unduh, dan nilai



Gambar 19. Menu Kelola untuk koordinator